Effect of `operator<=>` on the C++ Standard Library

Document Number: P0790R2
Date: 2019-01-21
Author: David Stone (davidmstone@google.com, david@doublewise.net)
Audience: LEWG, LWG

This paper lists (what are expected to be) non-controversial changes to the C++ standard library in response to P0515, which adds `operator<=>` to the language. This is expected to be non-controversial because it tries to match existing behavior as much as possible. As a result, all proposed additions are either `strong_equality` or `strong_ordering`, matching the existing comparison operators.

This document should contain a complete list of types or categories of types in C++.

Revision History

R1: A much broader version of this paper was presented to LEWG at a previous meeting. What remains in this paper is everything which the group did not find controversial and which probably does not require significant justification. All controversial aspects will be submitted in separate papers. R2: Added wording.

Backward Compatibility

The `operator<=>` proposal was written such that the "generated" operators are equivalent to source code rewrites – there is no actual `operator==` that a user could take the address of. Users are not allowed to form pointers to standard library member functions and are unable to form pointers to friend functions defined inline in the class. There are some cases where we do not specify how the operator was implemented, only that the expression `a @ b` is valid; these cases are not broken by such a change because users could not have depended on it, anyway. In general, we accept changes that overload existing functions, which also has the effect of breaking code which takes the address of a free function.

Types that are not proposed to get `operator<=>` in this paper

These types are not comparable now. This paper does not propose adding any new comparisons to any of these types.

- deprecated types
- exception types
- tag classes (`nothrow`, `piecewise_construct_t`, etc.)
- arithmetic function objects (`plus`, `minus`, etc.)
- comparison function objects (`equal_to`, etc.)
- `owner_less`
- logical function objects (`logical_and`, etc.)
- bitwise function objects (`bit_and`, etc.)
- `nested_exception`
- `allocator_traits`
- `pool_options`
- `char_traits`
- `iterator_traits`
- `numeric_limits`
- `pointer_traits`
- `regex_traits`
- `chrono::duration_values`
- `tuple_element`
- `max_align_t`
- `map::node_type`
- `map::insert_return_type`
- `set::node_type`
- `set::insert_return_type`
- `unordered_map::node_type`
- `unordered_map::insert_return_type`
- `unordered_set::node_type`

- unordered_set::insert_return_type
- any
- default_delete
- aligned_storage
- aligned_union
- system_clock
- steady_clock
- high_resolution_clock
- locale::facet
- locale::id
- ctype_base
- ctype
- ctype_byname
- codecvt_base
- codecvt
- codecvt_byname
- num_get
- num_put
- numpunct
- numpunct_byname
- collate
- collate_byname
- time_get
- time_get_byname
- time_put
- time_put_byname
- money_base
- money_get
- money_put
- money_punct
- moneypunct_byname
- message_base
- messages
- messages_byname
- FILE
- va_list
- back_insert_iterator
- front_insert_iterator
- insert_iterator
- ostream_iterator
- ostreambuf_iterator
- ios_base
- ios_base::Init
- basic_ios
- basic_streambuf
- basic_istream
- basic_iostream
- basic_ostream
- basic_stringbuf
- basic_istringstream
- basic_ostringstream
- basic_stringstream
- basic_filebuf
- basic_ifstream
- basic_ofstream
- basic_fstream
- gslice
- slice_array
- gslice_array
- mask_array
- indirect_array
- atomic_flag

- `thread`
- `mutex`
- `recursive_mutex`
- `timed_mutex`
- `timed_recursive_mutex`
- `lock_guard`
- `scoped_lock`
- `unique_lock`
- `once_flag`
- `shared_mutex`
- `shared_timed_mutex`
- `shared_lock`
- `condition_variable`
- `condition_variable_any`
- `promise`
- `future`
- `shared_future`
- `packaged_task`
- `random_device`
- `hash`
- `weak_ptr`
- `basic_regex`
- `sequential_execution_policy`
- `parallel_execution_policy`
- `parallel_vector_execution_policy`
- `default_searcher`
- `boyer_moore_searcher`
- `boyer_moore_horspool_searcher`
- `ratio`
- `integer_sequence`
- `seed_seq` (paper needed to add `strong_equality`)
- `enable_shared_from_this`: It would be nice to give it a `strong_ordering` to allow derived classes to `= default`. However, this means that all classes that do not explicitly delete their comparison operator get an `operator<=>` that compares only the `enable_shared_from_this` base class, which is almost certainly wrong. Since this is intended to be used as a base class, we should not add `operator<=>` to it. Moreover, classes which `enable_shared_from_this` are unlikely to be basic value classes so they do not lose much by not being able to default.
- `initializer_list`: `initializer_list` is a reference type. It would be strange to give it reference semantics on copy but value semantics for comparison. It would also be surprising if two `initializer_list` containing the same set of values compared as not equal. Therefore, I recommend not defining it for this type.

Types from C that are not proposed to get `operator<=>` in this paper

- `div_t`
- `ldiv_t`
- `lldiv_t`
- `imaxdiv_t`
- `timespec`
- `tm`
- `lconv`
- `fenv_t`
- `fpos_t`
- `mbstate_t`

Types that have only `==` and `!=`, and thus do not require `<=>`

- `type_info`
- `bitset` (paper would be needed to change this to `strong_ordering`)
- `allocator`
- `memory_resource`
- `synchronized_pool_resource`: (implicitly from `memory_resource` base class)
- `unsynchronized_pool_resource`: (implicitly from `memory_resource` base class)

- `monotonic_buffer_resource`: (implicitly from `memory_resource` base class)
- `polymorphic_allocator`
- `scoped_allocator_adaptor`
- `function` with `nullptr_t` only (no homogenous operator)
- `locale`
- `complex` (heterogeneous with `T` and homogeneous)
- `linear_congruential_engine`
- `mersenne_twister_engine`
- `subtract_with_carry_engine`
- `discard_block_engine`
- `independent_bits_engine`
- `shuffle_order_engine`
- `uniform_int_distribution`
- `uniform_int_distribution::param_type`
- `uniform_real_distribution`
- `uniform_real_distribution::param_type`
- `bernoulli_distribution`
- `bernoulli_distribution::param_type`
- `binomial_distribution`
- `binomial_distribution::param_type`
- `geometric_distribution`
- `geometric_distribution::param_type`
- `negative_binomial_distribution`
- `negative_binomial_distribution::param_type`
- `poisson_distribution`
- `poisson_distribution::param_type`
- `exponential_distribution`
- `exponential_distribution::param_type`
- `gamma_distribution`
- `gamma_distribution::param_type`
- `weibull_distribution`
- `weibull_distribution::param_type`
- `extreme_value_distribution`
- `extreme_value_distribution::param_type`
- `normal_distribution`
- `normal_distribution::param_type`
- `lognormal_distribution`
- `lognormal_distribution::param_type`
- `chi_squared_distribution`
- `chi_squared_distribution::param_type`
- `cauchy_distribution`
- `cauchy_distribution::param_type`
- `fisher_f_distribution`
- `fisher_f_distribution::param_type`
- `student_t_distribution`
- `student_t_distribution::param_type`
- `discrete_distribution`
- `discrete_distribution::param_type`
- `piecewsie_constant_distribution`
- `piecewsie_constant_distribution::param_type`
- `piecewise_linear_distribution`
- `piecewise_linear_distribution::param_type`
- `istream_iterator`
- `istreambuf_iterator`
- `filesystem::path::iterator`
- `filesystem::directory_iterator`
- `filesystem::recursive_directory_iterator`
- `match_results`
- `regex_iterator`
- `regex_token_iterator`
- `fpos`
- `forward_list::iterator`

- `list::iterator`
- `map::iterator`
- `set::iterator`
- `multimap::iterator`
- `multiset::iterator`
- `unordered_map::iterator`
- `unodered_set::iterator`
- `unordered_multimap::iterator`
- `unodered_multiset::iterator`

Types that should get `<=>` with a return type of `strong_ordering`, no change from current comparisons

These types are all currently comparable.

- `error_category`
- `error_code`
- `error_condition`
- `exception_ptr`
- `monostate`
- `chrono::duration`: heterogeneous with durations of other representations and periods
- `chrono::time_point`: heterogeneous in the duration
- `type_index`
- `filesystem::path`
- `filesystem::directory_entry`
- `thread::id`
- `array::iterator`
- `deque::iterator`
- `vector::iterator`
- `valarray::iterator`

Types that will get their `operator<=>` from a conversion operator

These types will get `operator<=>` if possible without any changes, just like they already have whatever comparison operators their underlying type has.

- `integral_constant` and all types deriving from `integral_constant` (has `operator T`)
- `bitset::reference` (has `operator bool`)
- `reference_wrapper` (has `operator T &`)
- `atomic` (has `operator T`)

This has the disadvantage that types which have a template comparison operator will not have their wrapper convertible. For instance, `std::reference_wrapper<std::string>` is not currently comparable. This does not affect `bitset::reference`, as it has a fixed conversion to `bool`, but it does affect the other three.

Types that wrap another type

- `array`
- `deque`
- `forward_list`
- `list`
- `vector` (including `vector<bool>`)
- `map`
- `set`
- `multimap`
- `multiset`
- `unordered_map`
- `unodered_set`
- `unodered_multimap`
- `unordered_multiset`

- `queue`
- `queue::iterator`
- `priority_queue`
- `priority_queue::iterator`
- `stack`
- `stack::iterator`
- `pair`
- `tuple`
- `reverse_iterator`
- `move_iterator`
- `optional`
- `variant`

This turned out to be much more complicated than expected and will require its own paper.

`basic_string`, `basic_string_view`, `char_traits`, and `sub_match`

Properly integrating `operator<=>` with these types requires more thought than this paper has room for, and thus will be discussed separately.

`unique_ptr` and `shared_ptr`

They contain state that is not observed in the comparison operators. Therefore, they will get their own paper.

valarray

Current comparison operators return a `valarray<bool>`, giving you the result for each pair (with undefined behavior for differently-sized `valarray` arguments). It might make sense to provide some sort of function that returns `valarray<comparison_category>`, but that should not be named `operator<=>`. This paper does not suggest adding `operator<=>` to `valarray`.

Types that have no comparisons now but are being proposed to get `operator<=>` in another paper

This paper does not propose changing any of the following types -- they are here only for completeness.

- `filesystem::file_status`
- `filesystem::space_info`
- `slice`
- `to_chars_result`
- `from_chars_result`

`nullptr_t`

Already supports `strong_equality` in the working draft. I will be writing a separate paper proposing `strong_ordering`.

Not Updating Concepts That Provide Comparisons

This category includes things like `BinaryPredicate` and `Compare`. This is addressed in a separate paper.

Not Updating Concepts That Require Comparisons

This includes things like `LessThanComparable` and `EqualityComparable`. This is addressed in a separate paper.

Miscellaneous

All `operator<=>` should be `constexpr` and `noexcept` where possible, following the lead of the language feature and allowing `= default` as an implementation strategy for some types.

When we list a result type as "unspecified" it is unspecified whether it has `operator<=>`. There are not any unspecified result types for which we currently guarantee any comparison operators are present, so there is no extra work to do here.

Wording

All wording is relative to N4791.

General wording

15.4.2.3 Operators [operators]

~~1 In this library, whenever a declaration is provided for an `operator!=`, `operator>`, `operator<=`, or `operator>=` for a type `T`, its requirements and semantics are as follows, unless explicitly specified otherwise.~~

```
bool operator!=(const T& x, const T& y);
```

2 Requires: Type `T` is `Cpp17EqualityComparable` (Table 23).

3 Returns: `!(x == y)`.

```
bool operator>(const T& x, const T& y);
```

4 Requires: Type `T` is `Cpp17LessThanComparable` (Table 24).

5 Returns: `y < x`.

```
bool operator<=(const T& x, const T& y);
```

6 Requires: Type `T` is `Cpp17LessThanComparable` (Table 24).

7 Returns: `!(y < x)`.

```
bool operator>=(const T& x, const T& y);
```

8 Requires: Type `T` is `Cpp17LessThanComparable` (Table 24).

9 Returns: `!(x < y)`.

1 Unless specified otherwise, if `lhs` and `rhs` are values of types from this library, the following shall hold:

- `lhs != rhs` is a valid expression if and only if `lhs == rhs` is a valid expression.
- `lhs < rhs`, `lhs > rhs`, `lhs <= rhs`, and `lhs >= rhs` are all valid expressions if and only if `lhs <=> rhs` is a valid expression.
- If `lhs <=> rhs` is a valid expression, `lhs == rhs` is a valid expression.

The requirements and semantics of these operators are as follows, unless explicitly specified otherwise.

Expression	Return type	Operational semantics
`lhs <=> rhs`	`std::strong_ordering`	If `lhs <=> rhs` yields `std::strong_ordering::equal`, the two objects have an equivalence relation as described in Cpp17EqualityComparable ([utility.arg.requirements]). Regardless of the return value, `lhs <=> rhs` is a total ordering relation (24.7).
`lhs == rhs`	Convertible to `bool`	`lhs <=> rhs == 0`
`lhs != rhs`	Convertible to `bool`	`!(lhs == rhs)`
`lhs < rhs`	Convertible to `bool`	`lhs <=> rhs < 0`
`lhs > rhs`	Convertible to `bool`	`lhs <=> rhs > 0`
`lhs <= rhs`	Convertible to `bool`	`lhs <=> rhs <= 0`
`lhs >= rhs`	Convertible to `bool`	`lhs <=> rhs >= 0`

`error_category`

18.5.2.3 Non-virtual members [syserr.errcat.nonvirtuals] constexpr error*category() noexcept; 1 Effects: Constructs an object of class error*category.

~~bool operator==(const error*category& rhs) const noexcept; 2 Returns: `this == &rhs`. bool operator!=(const error*~~category& rhs) ~~const noexcept; 3 Returns: `!(*this == rhs)`. bool operator<(const error_category& rhs) const noexcept; 4 Returns: `less<const error_category*>()(this, &rhs)`. [Note: `less` (19.14.7) provides a total ordering for pointers. — end note]~~ 18.5.2.4 Comparisons [syserr.errcat.comparisons]

For two values `lhs` and `rhs` of type `error_category`, the expression `lhs <=> rhs` is of type `std::strong_ordering`. If the address of `lhs` and `rhs` compare equal ([expr.eq]), `lhs <=> rhs` yields `std::strong_ordering::equal`; if `lhs` and `rhs` compare unequal, `lhs <=> rhs` yields `std::strong_ordering::less` if `std::less{}(&lhs, &rhs)` is `true` ([comparisons]), otherwise yields `std::strong_ordering::greater`.

18.5.2.~~4~~5 Program-defined classes derived from error_category [syserr.errcat.derived]

[...]

18.5.2.~~5~~6 Error category objects [syserr.errcat.objects]

`error_code` and `error_condition`

18.5.5 Comparison functions [syserr.compare] ~~bool operator==(const error_code& lhs, const error_code& rhs) noexcept;~~

1 Returns: `lhs.category() == rhs.category() && lhs.value() == rhs.value()`

```
bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
```

2 Returns: `lhs.category().equivalent(lhs.value(), rhs) || rhs.category().equivalent(lhs, rhs.value())`

```
bool operator==(const error_condition& lhs, const error_code& rhs) noexcept;
```

3 Returns: `rhs.category().equivalent(rhs.value(), lhs) || lhs.category().equivalent(rhs, lhs.value())`

```
bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
```

4 Returns: `lhs.category() == rhs.category() && lhs.value() == rhs.value()`

```
bool operator!=(const error_code& lhs, const error_code& rhs) noexcept;
bool operator!=(const error_code& lhs, const error_condition& rhs) noexcept;
bool operator!=(const error_condition& lhs, const error_code& rhs) noexcept;
bool operator!=(const error_condition& lhs, const error_condition& rhs) noexcept;
```

5 Returns: `!(lhs == rhs)`.

```
bool operator<(const error_code& lhs, const error_code& rhs) noexcept;
```

6 Returns: `lhs.category() < rhs.category() || (lhs.category() == rhs.category() && lhs.value() < rhs.value())`

```
bool operator<(const error_condition& lhs, const error_condition& rhs) noexcept;
```

7 Returns: `lhs.category() < rhs.category() || (lhs.category() == rhs.category() && lhs.value() < rhs.value())` For values `lhs_code` and `rhs_code` of type `error_code` and values `lhs_condition` and `rhs_condition` of type `error_condition`, the following holds:

1. The expression `lhs_code <=> rhs_code` is of type `std::strong_ordering`. Equivalent to `std::tie(lhs_code.category(), lhs_code.value()) <=> std::tie(rhs_code.category(), rhs_code.value())`.
2. The expression `lhs_condition <=> rhs_condition` is of type `std::strong_ordering`. Equivalent to `std::tie(lhs_condition.category(), lhs_condition.value()) <=> std::tie(rhs_condition.category(), rhs_condition.value())`.
3. The expression `lhs_code == rhs_condition` is of type `bool`. Equivalent to `lhs_code.category().equivalent(lhs_code.value(), rhs_condition) || rhs_condition.category().equivalent(lhs_code, rhs_category.value())`.

[Note: The function `tie` is defined in Clause 19.5. — end note]

`monostate`

19.7.9 monostate relational operators [variant.monostate.relops]

~~constexpr bool operator==(monostate, monostate) noexcept { return true; } constexpr bool operator!=(monostate, monostate) noexcept { return false; } constexpr bool operator<(monostate, monostate) noexcept { return false; } constexpr bool operator>(monostate, monostate) noexcept { return false; } constexpr bool operator<=(monostate, monostate) noexcept { return true; }~~

~~constexpr bool operator>=(monostate, monostate) noexcept { return true; }~~ For two values `lhs` and `rhs` of type `monostate`, the expression `lhs <=> rhs` is of type `std::strong_ordering` and yields `std::strong_ordering::equal`.

[Note: monostate objects have only a single state; they thus always compare equal. — end note]

`type_index`

19.17.2 type_index overview [type.index.overview]

```
namespace std {
    class type_index {
    public:
        type_index(const type_info& rhs) noexcept;
```

~~bool operator==(const type~~*~~index& rhs) const noexcept; bool operator!=(const type~~*~~index& rhs) const noexcept; bool operator< (const type~~*~~index& rhs) const noexcept; bool operator> (const type~~*~~index& rhs) const noexcept; bool operator<= (const type~~*~~index& rhs) const noexcept; bool operator>= (const type~~*~~index& rhs) const noexcept;~~ size*t hash*code() const noexcept; const char* name() const noexcept; private: const type_info* target; // exposition only // Note that the use of a pointer here, rather than a reference, // means that the default copy/move constructor and assignment // operators will be provided and work as expected. }; }

1 The class `type_index` provides a simple wrapper for `type_info` which can be used as an index type in associative containers (21.4) and in unordered associative containers (21.5).

19.17.3 `type_index` members [type.index.members]

```
type_index(const type_info& rhs) noexcept;
```

1 Effects: Constructs a `type_index` object, the equivalent of `target = &rhs`.

~~bool operator==(const type_index& rhs) const noexcept;~~

2 Returns: `*target == *rhs.target`.

```
bool operator!=(const type_index& rhs) const noexcept;
```

3 Returns: `*target != *rhs.target`.

```
bool operator<(const type_index& rhs) const noexcept;
```

4 Returns: `target->before(*rhs.target)`.

```
bool operator>(const type_index& rhs) const noexcept;
```

5 Returns: `rhs.target->before(*target)`.

```
bool operator<=(const type_index& rhs) const noexcept;
```

6 Returns: `!rhs.target->before(*target)`.

```
bool operator>=(const type_index& rhs) const noexcept;
```

7 Returns: `!target->before(*rhs.target)`.

```
size_t hash_code() const noexcept;
```

~~8~~2 Returns: `target->hash_code()`.

```
const char* name() const noexcept;
```

~~9~~2 Returns: `target->name()`.

19.17.4 Hash support [type.index.hash]

```
template<> struct hash<type_index>;
```

1 For an object `index` of type `type_index`, `hash<type_index>()(index)` shall evaluate to the same result as `index.hash_code()`.

19.17.5 Comparisons [type.index.comparisons]

For two values `lhs` and `rhs` of type `type_index`, the expression `lhs <=> rhs` is of type `std::strong_ordering`. If `*lhs.target == *rhs.target`, `lhs <=> rhs` yields `std::strong_ordering::equal`; if `lhs` and `rhs` compare unequal, `lhs <=> rhs` yields `std::strong_ordering::less` if `lhs.target->before(*rhs.target)` is `true`, otherwise yields `std::strong_ordering::greater`. [Note: A result of `std::strong_ordering::greater` happens only in the case where `rhs.target->before(*lhs.target)` is `true`. — end note]

Iterators

Amend the requirements table for `Cpp17RandomAccessIterator` (22.3.5.6) requirements:

| Expression | Return type | Operational semantics | Assertion/note pre-/post-condition | | ---------- | ---------- | ---------- | ---------- | | ~~a < b~~ | ~~contextually convertible to `bool`~~ | ~~`b - a > 0`~~ | ~~< is a total ordering relation~~ | | ~~a > b~~ | ~~contextually convertible to `bool`~~ | ~~b < a~~ | ~~> is a total ordering relation opposite to <~~ | | ~~a >= b~~ | ~~contextually convertible to `bool`~~ | ~~`!(a < b)`~~ | | | ~~a <= b~~ | ~~contextually convertible to `bool`~~ | ~~`!(a > b)`~~ | | | a <=> b | `std::strong_ordering` | | <=> is a total ordering relation |

`chrono::duration`

26.5.6 Comparisons [time.duration.comparisons] 1 In the function descriptions that follow, CT represents `common_type_t<A, B>`, where `A` and `B` are the types of the two arguments to the function.

~~template constexpr bool operator==(const duration& lhs, const duration& rhs); 2 Returns: `CT(lhs).count() == CT(rhs).count()`.~~

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator!=(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

3 Returns: `!(lhs == rhs)`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

4 Returns: `CT(lhs).count() < CT(rhs).count()`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

5 Returns: `rhs < lhs`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<=(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

6 Returns: `!(rhs < lhs)`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>=(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

7 Returns: `!(lhs < rhs)`. For two values `lhs` and `rhs`, each of which are instances of the class template `std::chrono::duration` (with possibly different `Rep` and `Period` template parameters), the expression `lhs <=> rhs` is equivalent to `CT(lhs).count() <=> CT(rhs).count()`. [Note: This implies the operation is `constexpr` if both `lhs` and `rhs` are `constexpr`. — end note]

`chrono::time_point`

26.6.6 Comparisons [time.point.comparisons]

~~template constexpr bool operator==(const time*point& lhs, const time*point& rhs); 1 Returns: `lhs.time_since_epoch() == rhs.time_since_epoch()`.~~

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator!=(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

2 Returns: `!(lhs == rhs)`.

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator<(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

3 Returns: `lhs.time_since_epoch() < rhs.time_since_epoch()`.

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator>(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

4 Returns: `rhs < lhs`. template constexpr bool operator<=(const time*point& lhs, const time*point& rhs); 5 Returns: `!(rhs < lhs)`.

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator>=(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

6 Returns: `!(lhs < rhs)`. For two values `lhs` and `rhs`, each of which are instances of the class template `std::chrono::time_point` (with possibly the same `Clock` template parameter but possibly different `Duration` template parameters), the expression `lhs <=> rhs` is equivalent to `CT(lhs).time_since_epoch() <=> CT(rhs).time_since_epoch()`. [Note: This implies the operation is `constexpr` if both `lhs` and `rhs` are `constexpr`. — end note]

`filesystem::path`

Modify the class synopsis (28.11.7 Class `path` [fs.class.path]) to remove the specification of comparison operators as friend functions:

```
namespace std::filesystem {
    class path {
    public:
        [...]
```

~~// 28.11.7.7, non-member operators friend bool operator==(const path& lhs, const path& rhs) noexcept; friend bool operator!=(const path& lhs, const path& rhs) noexcept; friend bool operator< (const path& lhs, const path& rhs) noexcept; friend bool operator<=(const path& lhs, const path& rhs) noexcept; friend bool operator> (const path& lhs, const path& rhs) noexcept; friend bool operator>=(const path& lhs, const path& rhs) noexcept; friend path operator/ (const path& lhs, const path& rhs);~~ [...] }; }

Add the following at the end of 28.11.7.4.8 [fs.path.compare]:

For two values `lhs` and `rhs` of type `std::filesystem::path`, the expression `lhs <=> rhs` is equivalent to `lhs.compare(rhs) <=> 0`

28.11.7.7 Non-member functions [fs.path.nonmember]

```
void swap(path& lhs, path& rhs) noexcept;
```

1 Effects: Equivalent to `lhs.swap(rhs)`.

```
size_t hash_value (const path& p) noexcept;
```

2 Returns: A hash value for the path `p`. If for two paths, `p1 == p2` then `hash_value(p1) == hash_value(p2)`.

~~friend bool operator==(const path& lhs, const path& rhs) noexcept;~~

3 Returns: `!(lhs < rhs) && !(rhs < lhs)`.

4 [Note: Path equality and path equivalence have different semantics. — (4.1) Equality is determined by the path non-member `operator==`, which considers the two paths' lexical representations only. [Example: `path("foo") == "bar"` is never `true`. — end example] — (4.2) Equivalence is determined by the `equivalent()` non-member function, which determines if two paths resolve (28.11.7) to the same file system entity. [Example: `equivalent("foo", "bar")` will be `true` when both paths resolve to the same file. — end example]

Programmers wishing to determine if two paths are "the same" must decide if "the same" means "the same representation" or "resolve to the same actual file", and choose the appropriate function accordingly. — end note]

```
friend bool operator!=(const path& lhs, const path& rhs) noexcept;
```

5 Returns: `!(lhs == rhs)`.

```
friend bool operator< (const path& lhs, const path& rhs) noexcept;
```

6 Returns: `lhs.compare(rhs) < 0`.

```
friend bool operator<=(const path& lhs, const path& rhs) noexcept;
```

7 Returns: `!(rhs < lhs)`.

```
friend bool operator> (const path& lhs, const path& rhs) noexcept;
```

8 Returns: `rhs < lhs`.

```
friend bool operator>=(const path& lhs, const path& rhs) noexcept;
```

9 Returns: `!(lhs < rhs)`.

```
friend path operator/ (const path& lhs, const path& rhs);
```

~~10~~3 Effects: Equivalent to: `return path(lhs) /= rhs;`

`filesystem::directory_entry`

Amend the class synopsis:

```
namespace std::filesystem {
    class directory_entry {
    public:
        [...]
```

~~bool operator==(const directory_entry& rhs) const noexcept; bool operator!=(const directory_entry& rhs) const noexcept; bool operator< (const directory_entry& rhs) const noexcept; bool operator<=(const directory_entry& rhs) const noexcept; bool operator> (const directory_entry& rhs) const noexcept; bool operator>=(const directory_entry& rhs) const noexcept;~~

```
        [...]
    };
}
```

Amend 28.11.11.3:

~~bool operator==(const directory_entry& rhs) const noexcept;~~

31 Returns: `pathobject == rhs.pathobject`.

```
bool operator!=(const directory_entry& rhs) const noexcept;
```

32 Returns: `pathobject != rhs.pathobject`.

```
bool operator< (const directory_entry& rhs) const noexcept;
```

33 Returns: `pathobject < rhs.pathobject`.

```
bool operator> (const directory_entry& rhs) const noexcept;
```

34 Returns: `pathobject > rhs.pathobject`.

```
bool operator<=(const directory_entry& rhs) const noexcept;
```

35 Returns: `pathobject <= rhs.pathobject`.

```
bool operator>=(const directory_entry& rhs) const noexcept;
```

36 Returns: `pathobject >= rhs.pathobject`.

28.11.11.4 Comparisons [fs.dir.entry.comparisons]

For two values `lhs` and `rhs` of type `std::filesystem::directory_entry`, the expression `lhs <=> rhs` is equivalent to `lhs.pathobject <=> rhs.pathobject`.

thread::id

31.3.2.1 Class thread::id [thread.thread.id]

```
namespace std {
    class thread::id {
    public:
        id() noexcept;
    };
```

~~bool operator==(thread::id x, thread::id y) noexcept; bool operator!=(thread::id x, thread::id y) noexcept; bool operator<(thread::id x, thread::id y) noexcept; bool operator>(thread::id x, thread::id y) noexcept; bool operator<=(thread::id x, thread::id y) noexcept; bool operator>=(thread::id x, thread::id y) noexcept;~~ template basic*ostream& operator*<<*(basic*ostream& out, thread::id id); // hash support template struct hash; template<> struct hash; }

1 An object of type `thread::id` provides a unique identifier for each thread of execution and a single distinct value for all `thread` objects that do not represent a thread of execution (31.3.2). Each thread of execution has an associated `thread::id` object that is not equal to the `thread::id` object of any other thread of execution and that is not equal to the `thread::id` object of any `thread` object that does not represent threads of execution.

2 `thread::id` is a trivially copyable class (10.1). The library may reuse the value of a `thread::id` of a terminated thread that can no longer be joined.

3 [Note: Relational operators allow `thread::id` objects to be used as keys in associative containers. — end note]

```
id() noexcept;
```

4 Effects: Constructs an object of type `id`.

5 Ensures: The constructed object does not represent a thread of execution.

~~bool operator==(thread::id x, thread::id y) noexcept;~~

6 Returns: `true` only if `x` and `y` represent the same thread of execution or neither `x` nor `y` represents a thread of execution.

```
bool operator!=(thread::id x, thread::id y) noexcept;
```

7 Returns: `!(x == y)`

```
bool operator<(thread::id x, thread::id y) noexcept;
```

8 Returns: A value such that `operator<` is a total ordering as described in 24.7.

```
bool operator>(thread::id x, thread::id y) noexcept;
```

9 Returns: `y < x`.

```
bool operator<=(thread::id x, thread::id y) noexcept;
```

10 Returns: `!(y < x)`.

```
bool operator>=(thread::id x, thread::id y) noexcept;
```

11 Returns: `!(x < y)`.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<< (basic_ostream<charT, traits>& out, thread::id id);
```

~~12~~6 Effects: Inserts an unspecified text representation of `id` into `out`. For two objects of type `thread::id` `x` and `y`, if `x == y` the `thread::id` objects have the same text representation and if `x != y` the `thread::id` objects have distinct text representations.

~~13~~7 Returns: `out`.

```
template<> struct hash<thread::id>;
```

~~14~~8 The specialization is enabled (19.14.18).

For two values `lhs` and `rhs` of type `std::thread::id`, the expression `lhs <=> rhs` is of type `std::strong_ordering`. If `lhs` and `rhs` represent the same thread of execution or neither `x` nor `y` represents a thread of execution, `lhs <=> rhs` yields `std::strong_ordering::equal`; if `lhs` and `rhs` compare unequal, `lhs <=> rhs` yields a value consistent with a total order as described in 24.7.